LAPORAN PRAKTIKUM

MATAKULIAH PRAKTIKUM ALGORITMA DAN STRUKTUR DATA

“ANALISIS KOMPLEKSITAS ALGORITMA”

Disusun oleh :

TSABITA NABIGHA LUBNAYYA

NIM	:	2025573010095
Kelas	:	TI-1D
Jurusan	:	Teknologi Informasi dan Komputer
PRODI	:	Teknik Informatika
Dosen Pengampu	:	Muhammad Reza Zulman, S.S.T., M.Sc.

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI LHOKSEUMAWE

2026

LEMBAR PENGESAHAN

No. Praktikum	:	05/TIK/TI-1D/ALGORITMA-DAN-STRUKTUR-DATA/2026
Nama Praktikum	:	Praktikum Algoritma dan Sturktur Data
Pembuat Laporan	:	Tsabita Nabigha LubNayya
NIM	:	2025573010095
Kelas	:	TI-1D
Jurusan	:	Teknologi Informasi dan Komputer
PRODI	:	Teknik Informatika
Tanggal Praktikum	:	23 April 2026
Tanggal Penyerahan Laporan	:	29 April 2026

NO.	DESKRIPSI REVIEW	TANGGAL

Mengetahui

Dosen Pengampu,

Muhammad Reza Zulman, S.S.T., M.Sc.

NIP. 199205012022031005

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, laporan praktikum Struktur Data dan Algoritma dengan judul *"Analisis Kompleksitas Algoritma"* ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas praktikum serta untuk memahami konsep penting dalam analisis efisiensi algoritma, khususnya Big O Notation, Time Complexity, dan Space Complexity.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun.

Lhokseumawe, 28 April 2026

Penulis

TSABITA NABIGHA LUBNAYYA

NIM. 2025573010095

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii

BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 1
1.3 Tujuan Praktikum..…….. 1

BAB II LANDASAN TEORI 2
2.1 Big O Notation 2
2.2 Time Complexity 2
2.3 Space Complexity 2

BAB III METODOLOGI 3
3.1 Tools yang Digunakan 3
3.2 Langkah-langkah Praktikum 3

BAB IV HASIL DAN PEMBAHASAN 13

BAB V PENUTUP 14
5.1 Kesimpulan 14
5.2 Saran 14

BAB I

PENDAHULUAN

1.1. Latar Belakang

Dalam pengolahan data, efisiensi algoritma menjadi faktor penting. Algoritma yang tidak efisien akan menyebabkan waktu eksekusi yang lama, terutama pada data berskala besar.

Untuk mengatasi hal tersebut, digunakan analisis kompleksitas algoritma dengan Big O Notation yang dapat menggambarkan pertumbuhan waktu dan memori terhadap ukuran input.

1.2. Rumusan Masalah

Adapun rumusan masalah pada praktikum ini adalah:

1. Apa itu Big O Notation?
2. Apa itu Time dan Space Complexity?
3. Bagaimana menganalisis efisiensi algoritma?

1.3. Tujuan Praktikum

Tujuan dari praktikum ini adalah:

1. Memahami konsep kompleksitas
2. Menganalisis algoritma
3. Membandingkan performa algoritma

BAB II

DASAR TEORI

2.1. Big O Notation

Big O digunakan untuk mengukur pertumbuhan performa algoritma.
Jenis-jenis:

- $O(1) \rightarrow$ Konstan
- $O(\log n) \rightarrow$ Logaritmik
- $O(n) \rightarrow$ Linear
- $O(n \log n) \rightarrow$ Linearitmik
- $O(n^2) \rightarrow$ Kuadratik
- $O(2^n) \rightarrow$ Eksponensial

2.2. Class dan OOP

Mengukur waktu eksekusi algoritma.

2.3. Space Complexity

Mengukur penggunaan memori tambahan.

BAB III

METODOLOGI PRAKTIKUM

3.1. Alat dan Bahan

Alat dan bahan yang digunakan dalam praktikum ini adalah:

1. Laptop atau komputer
2. Aplikasi *Git Bash.*
3. *GitHub account*
4. *Visual Studio Code.*
5. Koneksi *Internet.*

3.2. Langkah-langkah Praktikum

Adapun langkah-langkah yang dilakukan dalam praktikum ini adalah sebagai berikut:

3.2.1 Persiapan *Project.*

3.2.1.1. Menavigasikan *Git Bash* ke folder *repository* yang sudah di-*clone,* lalu buat folder baru Bernama Praktikum-5.

```
tasbi@DESKTOP-U96FG97 MINGW64 /d/Praktikum_ASD
$ cd /d/Praktikum_ASD/2025573010095_StrukturDatadanAlgoritma

tasbi@DESKTOP-U96FG97 MINGW64 /d/Praktikum_ASD/2025573010095_StrukturDatadanAlgo
ritma (main)
$ mkdir praktikum-5

tasbi@DESKTOP-U96FG97 MINGW64 /d/Praktikum_ASD/2025573010095_StrukturDatadanAlgo
ritma (main)
$ cd praktikum-5
```

Gambar 1. Git Bash dinavigasikan ke *folder repository* dan membuat *folder* baru.

3.2.1.2. Inisialisasi NPM Project.

```
tasbi@DESKTOP-U96FG97 MINGW64 /d/Praktikum_ASD/2025573010095_StrukturDatadanAlgo
ritma/praktikum-5 (main)
$ npm init -y
Wrote to D:\Praktikum_ASD\2025573010095_StrukturDatadanAlgoritma\praktikum-5\pac
kage.json:

{
  "name": "praktikum-5",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Gambar 2. Inisililsasi npm *project.*

3.2.2. Intro Kompleksitas

3.2.2.1. Membuat *file* 01-intro-kompleksitas.js .

```
tasbi@DESKTOP-U96FG97 MINGW64 /d/Praktikum_ASD/2025573010095_StrukturDatadanAlgo
ritma/praktikum-5 (main)
$ touch 01-intro-kompleksitas.js
```

Gambar 3. Membuat *file* Intro Kompleksitas.

3.2.2.2. Mengetikkan kode contoh pada modul ke dalam *file* 01-intro-kompleksitas.js di *VS Code.*

```
JS 01-intro-kompleksitas.js > ...
function ukurWaktu(label, fn) {
 const mulai = Date.now();
 fn();
 const selesai = Date.now();
 console.log(`${label}: ${selesai - mulai} ms`);
}
const N = 100_000;
function jumlahkanLinear(n) {
 let total = 0;
 for (let i = 1; i <= n; i++) total += i;
 return total;
}
function jumlahkanRumus(n) {
 return (n * (n + 1)) / 2;
}
function cariPasangan(arr) {
 const pasangan = [];
 for (let i = 0; i < arr.length; i++) {
 for (let j = i + 1; j < arr.length; j++) {
 if (arr[i] + arr[j] === 10) pasangan.push([arr[i], arr[j]]);
 }
 }
 return pasangan;
}
const data = Array.from({length: 5000}, (_, i) => i);
console.log('=== Perbandingan Waktu Eksekusi ===');
ukurWaktu('O(1) jumlahkanRumus ', () => jumlahkanRumus(N));
ukurWaktu('O(n) jumlahkanLinear ', () => jumlahkanLinear(N));
ukurWaktu('O(n²) cariPasangan ', () => cariPasangan(data));
console.log('\nHasil sama?', jumlahkanLinear(100) === jumlahkanRumus(100));
```

```
    Directory:
    D:\Praktikum_ASD\2025573010095_St


Mode                 LastWriteTime
----                 -------------
d-----        4/29/2026   3:13 PM


PS D:\Praktikum_ASD\2025573010095_Str



PS D:\Praktikum_ASD\2025573010095_StrukturDatadanAlgoritma
\praktikum-5> node 01-intro-kompleksitas.js
=== Perbandingan Waktu Eksekusi ===
O(1) jumlahkanRumus : 0 ms
O(n) jumlahkanLinear : 2 ms
O(n²) cariPasangan : 38 ms

Hasil sama? true
PS D:\Praktikum_ASD\2025573010095_StrukturDatadanAlgoritma
\praktikum-5>
```

Gambar 4. *Input* dan *Output* dari *running program code* melalui *VS Code.*

3.2.3. Kelas Big O.

3.2.3.1. Buat *file* 02-kelas-big-o.js

```
tasbi@DESKTOP-U96FG97 MINGW64 /d/Praktikum_ASD/2025573010095_StrukturDatadanAlgo
ritma/praktikum-5 (main)
$ touch 02-kelas-big-o.js
```

Gambar 5. Membuat file Kelas Big-O.js

3.2.3.2. Mengetikkan kode contoh pada modul ke dalam *file* 02-kelas-big-o .js di *VS Code.*

```
02-kelas-big-o.js > ...
function ambilPertama(arr) { return arr[0]; }
function tambahkanItem(arr, item) { arr.push(item); }
function isGenap(n) { return n % 2 === 0; }
function binarySearch(arr, target) {
 let kiri = 0, kanan = arr.length - 1;
 let langkah = 0;
 while (kiri <= kanan) {
 langkah++;
 const tengah = Math.floor((kiri + kanan) / 2);
 if (arr[tengah] === target) {
     console.log(` Ditemukan di indeks ${tengah} setelah ${langkah} langkah`);
     return tengah;}
 if (arr[tengah] < target) kiri = tengah + 1;
    else kanan = tengah - 1;}
    return -1;}
function cariMax(arr) {
     let maks = arr[0];
       for (const x of arr) if (x > maks) maks = x;
       return maks;}
function bubbleSortNaif(arr) {
 const a = [...arr];
     for (let i = 0; i < a.length; i++)
         for (let j = 0; j < a.length - i - 1; j++)
             if (a[j] > a[j+1]) [a[j], a[j+1]] = [a[j+1], a[j]];
             return a;}
function fibRekursif(n) {
 if (n <= 1) return n;
 return fibRekursif(n-1) + fibRekursif(n-2);}
console.log('=== O(1) — selalu cepat ===');
console.log(ambilPertama([10,20,30,40,50]));
console.log(isGenap(42));
console.log('\n=== O(log n) — Binary Search ===');
const sorted = Array.from({length:1_000_000},(_,i)=>i);
binarySearch(sorted, 731_452);
console.log('\n=== O(n) — Linear Search ===');
console.log('Max dari 1000 elemen:',
cariMax(Array.from({length:1000},()=>Math.random()*1000|0)));
console.log('\n=== O(n²) — Bubble Sort (kecil saja) ===');
console.log(bubbleSortNaif([64, 34, 25, 12, 22, 11, 90]));
console.log('\n=== O(2ⁿ) — Fibonacci Rekursif (n kecil!) ===');
    for (let i = 0; i <= 10; i++) process.stdout.write(fibRekursif(i) + ' ');
console.log('');
console.log('\nWaktu fib(35) O(2ⁿ):');
let t = Date.now(); fibRekursif(35); console.log(Date.now()-t, 'ms');
console.log('Waktu fib(35) O(n) memoization:');
const memo = {};
function fibMemo(n) {
     if (n <= 1) return n;
        if (memo[n]) return memo[n];
        return memo[n] = fibMemo(n-1) + fibMemo(n-2);}
t = Date.now(); fibMemo(35); console.log(Date.now()-t, 'ms');node
```

```
PS D:\Praktikum_ASD\2025573010095_StrukturDatadanAlgoritma
\praktikum-5> node 02-kelas-big-o.js
=== O(1) — selalu cepat ===
10
true

=== O(log n) — Binary Search ===
 Ditemukan di indeks 731452 setelah 19 langkah

=== O(n) — Linear Search ===
Max dari 1000 elemen: 999

=== O(n²) — Bubble Sort (kecil saja) ===
[
  11, 12, 22, 25,
  34, 64, 90
]

=== O(2ⁿ) — Fibonacci Rekursif (n kecil!) ===
0 1 1 2 3 5 8 13 21 34 55

Waktu fib(35) O(2ⁿ):
133 ms
Waktu fib(35) O(n) memoization:
0 ms
[redacted]
```

Gambar 6. Tampilan *input & output code* pada VS Code.

3.2.4. Space Complexity

3.2.4.1. Buat *file* 03-space-complexity.js

```
tasbi@DESKTOP-U96FG97 MINGW64 /d/Praktikum_ASD/2025573010095_StrukturDatadanAlgo
ritma/praktikum-5 (main)
$ touch 03-space-complexity.js
```

Gambar 7. Membuat *file* Class Dasar pada Git Bash.

3.2.4.2. Mengetikkan kode contoh pada modul ke dalam *file* 03-space-complexity.js di *VS Code*

```
JS 03-space-complexity.js > hitungUnik
function jumlahArray(arr) {
 let total = 0;
 for (const x of arr) total += x;
 return total;
}
function duplikasiArray(arr) {
 const baru = [];
 for (const x of arr) baru.push(x * 2);
 return baru;
}
function faktorialRekursif(n) {
 if (n <= 1) return 1;
 return n * faktorialRekursif(n - 1);
}
function faktorialIteratif(n) {
 let hasil = 1;
 for (let i = 2; i <= n; i++) hasil *= i;
 return hasil;
}
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
console.log('Jumlah array :', jumlahArray(arr));
console.log('Duplikasi array:', duplikasiArray(arr));
console.log('Faktorial 10 rekursif :', faktorialRekursif(10));
console.log('Faktorial 10 iteratif :', faktorialIteratif(10));
function hitungUnik(arr) {
 const seen = new Set();
  for (const x of arr) seen.add(x);
 return seen.size;
}
const dataAcak = [1,2,3,2,1,4,5,3,6,4,7];
console.log('Elemen unik:', hitungUnik(dataAcak));
```

```
\praktikum-5>
PS D:\Praktikum_ASD\2025573010095_StrukturDatadanAlgoritma
\praktikum-5> node 03-space-complexity.js
Jumlah array : 55
Duplikasi array: [
   2,  4,  6,  8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 7
```

Gambar 9. Tampilan input dan output *code* pada VS Code.

3.2.5. Refaktor Kode yang Lambat.

3.2.5.1. Buat *file* 04-refactor.js.

```
tasbi@DESKTOP-U96FG97 MINGW64 /d/Praktikum_ASD/2025573010095_StrukturDatadanAlgo
ritma/praktikum-5 (main)
$ touch 04-refactor.js
```

Gambar 10. Membuat *file* Inheritance pada Git Bash.

3.2.5.2. Mengetikkan kode contoh pada modul ke dalam *file* touch 04-refactor.js di *VS Code*

```js
JS 04-refactor.js > cariPasanganLambat
function adaDuplikatLambat(arr) {
 for (let i = 0; i < arr.length; i++)
 for (let j = i+1; j < arr.length; j++)
 if (arr[i] === arr[j]) return true;
 return false;
}
function adaDuplikatCepat(arr) {
 const seen = new Set();
 for (const x of arr) {
 if (seen.has(x)) return true;
 seen.add(x);
 }
 return false;
}
function frekuensiLambat(arr) {
 const unik = [];
 const hitung = [];
 for (const x of arr) {
 const idx = unik.indexOf(x);
 if (idx === -1) { unik.push(x); hitung.push(1); }
 else hitung[idx]++;
 }
 return Object.fromEntries(unik.map((u,i) => [u, hitung[i]]));}
function frekuensiCepat(arr) {
 const counter = {};
 for (const x of arr) counter[x] = (counter[x] || 0) + 1;
 return counter;
}
const besar = Array.from({length: 20000}, () => Math.floor(Math.random() * 100
let t = Date.now();
adaDuplikatLambat(besar);
console.log('Duplikat LAMBAT O(n²):', Date.now()-t, 'ms');
t = Date.now();
adaDuplikatCepat(besar);
console.log('Duplikat CEPAT O(n) :', Date.now()-t, 'ms');
const kata = ['a','b','a','c','b','a','d'];
console.log('\nFrekuensi:', frekuensiCepat(kata));
console.log('\n=== Latihan 3.3 ===\n');
function cariPasanganLambat(arr, target) {
    for (let i = 0; i < arr.length; i++) {
        for (let j = i + 1; j < arr.length; j++) {
            if (arr[i] + arr[j] === target) {
                return [arr[i], arr[j]];
            }}}
    return null;}
```

```js
JS 04-refactor.js > cariPasanganLambat
function cariPasanganCepat(arr, target) {
    const seen = new Set();
    for (let num of arr) {
        const pasangan = target - num;
        if (seen.has(pasangan)) {
            return [pasangan, num];}
        seen.add(num);}
    return null;}
const data = [2, 7, 11, 15];
const target = 9;
console.log("=== Uji Dasar ===");
console.log("Lambat:", cariPasanganLambat(data, target));
console.log("Cepat :", cariPasanganCepat(data, target));
function ukurWaktu(label, fn) {
    const start = Date.now();
    fn();
    const end = Date.now();
    console.log(`${label}: ${end - start} ms`);}
const dataBesar = Array.from({ length: 50000 }, () =>
    Math.floor(Math.random() * 100000)
);
const targetBesar = -1;
console.log("\n=== Uji Performa ===");
ukurWaktu("Lambat O(n^2)", () => {
    cariPasanganLambat(dataBesar, targetBesar);
});
ukurWaktu("Cepat  O(n)", () => {
    cariPasanganCepat(dataBesar, targetBesar);
});
```

```
\praktikum-5> node 04-refactor.js
Duplikat LAMBAT O(n²): 0 ms
Duplikat CEPAT O(n) : 0 ms

Frekuensi: { a: 3, b: 2, c: 1, d: 1 }

=== Latihan 3.3 ===

=== Uji Dasar ===
Lambat: [ 2, 7 ]
Cepat : [ 2, 7 ]

=== Uji Performa ===
Lambat O(n^2): 2672 ms
Cepat  O(n): 7 ms
```

Gambar 11. Tampilan input dan output *code* pada VS Code.

3.2.5 Latihan dan Tugas.

3.2.6.1. Buat folder tugas di dalam praktikum-5/ lalu buat *file* tugas-1.js, tugas-2.js, dan latihan-1.

```
tasbi@DESKTOP-U96FG97 MINGW64 /d/Praktikum_ASD/2025573010095_StrukturDatadanAlgo
ritma/praktikum-5 (main)
$ mkdir tugas

tasbi@DESKTOP-U96FG97 MINGW64 /d/Praktikum_ASD/2025573010095_StrukturDatadanAlgo
ritma/praktikum-5 (main)
$ touch tugas/tugas-1.js tugas/tugas-2.js

tasbi@DESKTOP-U96FG97 MINGW64 /d/Praktikum_ASD/2025573010095_StrukturDatadanAlgo
ritma/praktikum-5 (main)
$ touch latihan-1.js
```

Gambar 13. Membuat *file* tugas-1.js, tugas-2.js, dan latihan-1 pada Git Bash.

3.2.6.2. Kode Latihan-1.js di *VS Code*

```
latihan-1.js > ...
// fungsi A
function fn(n) {
    return n * 2;
}
// fungsi B
function fungsiB(n) {
    for (let i = 0; i < n; i++) {
        for (let j = 0; j < n; j++) {
        }
    }
}
// fungsi C
function fungsiC(n) {
    for (let i = 1; i < n; i *= 2) {
    }
}
// fungsi D
function fungsiD(n) {
    const arr = Array.from({ length: n }, (_, i) => i);

    arr.forEach(() => {
        arr.forEach(() => {
            arr.forEach(() => {
            });
        });
    });
}
// hitung waktu
function hitungKompleksitas(n, fn, nama) {
    const mulai = Date.now();
    fn(n);
    const selesai = Date.now();
    console.log(nama + " -> " + (selesai - mulai) + " ms");
}
// run
const n = 1000;
hitungKompleksitas(n, fn, "fungsi A");
hitungKompleksitas(n, fungsiB, "fungsi B");
hitungKompleksitas(n, fungsiC, "fungsi C");
hitungKompleksitas(50, fungsiD, "fungsi D");
```

```
Filter (e.g. text, !exclude, \escape)
946                     latihan-1.js:41
947                     latihan-1.js:41
948                     latihan-1.js:41
949                     latihan-1.js:41
950                     latihan-1.js:41
951                     latihan-1.js:41
952                     latihan-1.js:41
953                     latihan-1.js:41
954                     latihan-1.js:41
955                     latihan-1.js:41
956                     latihan-1.js:41
957                     latihan-1.js:41
958                     latihan-1.js:41
959                     latihan-1.js:41
960                     latihan-1.js:41
961                     latihan-1.js:41
962                     latihan-1.js:41
963                     latihan-1.js:41
964                     latihan-1.js:41
965                     latihan-1.js:41
966                     latihan-1.js:41
967                     latihan-1.js:41
968                     latihan-1.js:41
969                     latihan-1.js:41
970                     latihan-1.js:41
971                     latihan-1.js:41
972                     latihan-1.js:41
973                     latihan-1.js:41
974                     latihan-1.js:41
975                     latihan-1.js:41
976                     latihan-1.js:41
977                     latihan-1.js:41
978                     latihan-1.js:41
979                     latihan-1.js:41
980                     latihan-1.js:41
981                     latihan-1.js:41
982                     latihan-1.js:41
983                     latihan-1.js:41
984                     latihan-1.js:41
985                     latihan-1.js:41
986                     latihan-1.js:41
987                     latihan-1.js:41
988                     latihan-1.js:41
989                     latihan-1.js:41
990                     latihan-1.js:41
991                     latihan-1.js:41
992                     latihan-1.js:41
993                     latihan-1.js:41
994                     latihan-1.js:41
995                     latihan-1.js:41
996                     latihan-1.js:41
997                     latihan-1.js:41
998                     latihan-1.js:41
999                     latihan-1.js:41
Waktu eksekusi: 286 ms  latihan-1.js:36
```

Gambar 14. Tampilan input dan output *code* pada VS Code.

3.2.6.2. Code Tugas-1.js Analisis dan Refactor

```javascript
function ukurWaktu(label, fn) {
    const start = Date.now();
    fn();
    const end = Date.now();
    console.log(`${label}: ${end - start} ms`);
}
console.log("=== FUNGSI A ===");
const a1 = [1,2,3,4,5];
const a2 = [3,4,5,6,7];
console.log("Lambat:", intersectionLambat(a1, a2));
console.log("Cepat :", intersectionCepat(a1, a2));
console.log("\n=== FUNGSI B ===");
const words = ['eat','tea','tan','ate','nat','bat'];
console.log(kelompokAnagram(words));
console.log("\n=== FUNGSI C ===");
const nums = [2,3,5,7];
console.log("Lambat:", sumKuadratLambat(nums));
console.log("Cepat :", sumKuadratCepat(nums));
const besar1 = Array.from({length: 5000}, () => Math.floor(Math.random()*1000));
const besar2 = Array.from({length: 5000}, () => Math.floor(Math.random()*1000));
console.log("\n=== UJI PERFORMA ===");
ukurWaktu("Intersection Lambat O(n^2)", () => {
    intersectionLambat(besar1, besar2);
});
ukurWaktu("Intersection Cepat O(n)", () => {
    intersectionCepat(besar1, besar2);
});
ukurWaktu("Sum Kuadrat Lambat O(n^3)", () => {
    sumKuadratLambat(besar1.slice(0, 200));
});
ukurWaktu("Sum Kuadrat Cepat O(n log n)", () => {
    sumKuadratCepat(besar1.slice(0, 200));
});

// === kesimpulan ===
// - Versi cepat selalu lebih baik untuk data besar
// - Menggunakan Set / sorting bisa mengurangi kompleksitas drastis
// - Trade-off: versi cepat kadang butuh memori tambahan
```

Gambar 15. Tampilan input dan output *code* pada VS Code.

3.2.6.3. Buka *file* tugas-2.js Visualisasi Pertembuhan Big O dan masukkan code tugasnya pada VS Code.

```
tugas > JS tugas-2.js > fn_O1
function fn_O1(n) {
    return n + 1;
}
function fn_On(n) {
    let total = 0;
    for (let i = 0; i < n; i++) {
        total += i;
    }
    return total;
}
function fn_OnLogn(n) {
    let total = 0;
    for (let i = 0; i < n; i++) {
        for (let j = 1; j < n; j *= 2) {
            total += j;
        }  }
    return total;
}
function fn_On2(n) {
    let total = 0;

    for (let i = 0; i < n; i++) {
        for (let j = 0; j < n; j++) {
            total += i + j;
        }}
    return total;
}
function ukurWaktu(fn, n) {
    const start = Date.now();
    fn(n);
    const end = Date.now();
    return end - start;
}
function benchmarkSemua(ukuranData) {
    console.log("=== Benchmark Big O ===");

    ukuranData.forEach(n => {
        console.log(`\nUkuran data: ${n}`);

        const t1 = ukurWaktu(fn_O1, n);
        const t2 = ukurWaktu(fn_On, n);
        const t3 = ukurWaktu(fn_OnLogn, n);
        const t4 = ukurWaktu(fn_On2, n);

        console.log(`O(1)     : ${t1} ms`);
        console.log(`O(n)     : ${t2} ms`);
        console.log(`O(n log n): ${t3} ms`);
        console.log(`O(n^2)   : ${t4} ms`);
    });
}
benchmarkSemua([100, 500, 1000, 5000, 10000]);
// === analisis dan kesimpulan ===
// - O(1) hampir tidak berubah walaupun ukuran data meningkat
// - O(n) meningkat secara linear
// - O(n log n) lebih lambat dari O(n), tapi masih cukup efisien
// - O(n^2) meningkat sangat drastis dan menjadi paling lambat

// Kesimpulan:
// Pola pertumbuhan yang diamati konsisten dengan teori Big O,
// dimana semakin tinggi kompleksitas, semakin cepat waktu eksekusi meningkat.
```

```
Filter (e.g. text, !exclude, \escape)

D:\Program Files\nodejs\node.exe .\tugas\tugas-2.js
=== Benchmark Big O ===                tugas-2.js:35
                                       tugas-2.js:38
Ukuran data: 100                       tugas-2.js:38
O(1)     : 0 ms                        tugas-2.js:45
O(n)     : 0 ms                        tugas-2.js:46
O(n log n): 0 ms                       tugas-2.js:47
O(n^2)   : 3 ms                        tugas-2.js:48
                                       tugas-2.js:38
Ukuran data: 500                       tugas-2.js:38
O(1)     : 0 ms                        tugas-2.js:45
O(n)     : 0 ms                        tugas-2.js:46
O(n log n): 1 ms                       tugas-2.js:47
O(n^2)   : 7 ms                        tugas-2.js:48
                                       tugas-2.js:38
Ukuran data: 1000                      tugas-2.js:38
O(1)     : 0 ms                        tugas-2.js:45
O(n)     : 0 ms                        tugas-2.js:46
O(n log n): 0 ms                       tugas-2.js:47
O(n^2)   : 4 ms                        tugas-2.js:48
                                       tugas-2.js:38
Ukuran data: 5000                      tugas-2.js:38
O(1)     : 0 ms                        tugas-2.js:45
O(n)     : 0 ms                        tugas-2.js:46
O(n log n): 1 ms                       tugas-2.js:47
O(n^2)   : 86 ms                       tugas-2.js:48
                                       tugas-2.js:38
Ukuran data: 10000                     tugas-2.js:38
O(1)     : 0 ms                        tugas-2.js:45
O(n)     : 0 ms                        tugas-2.js:46
O(n log n): 1 ms                       tugas-2.js:47
O(n^2)   : 134 ms                      tugas-2.js:48
```

Gambar 16. Tampilan input dan output *code* pada VS Code.

3.2.6 Submit ke GitHub

3.2.9.1. Kembali ke root folder repository pada GitHub lalu cek status.

```
tasbi@DESKTOP-U96FG97 MINGW64 /d/Praktikum_ASD/2025573010095_StrukturDatadanAlgoritma/praktikum-5 (main
$ cd ..

tasbi@DESKTOP-U96FG97 MINGW64 /d/Praktikum_ASD/2025573010095_StrukturDatadanAlgoritma (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        praktikum-5/

nothing added to commit but untracked files present (use "git add" to track)
```

Gambar 17. Tampilan Kembali ke root repository di Git Bash

3.2.9.2. Menambahkan seluruh folder praktikum-4 ke repository.

```
tasbi@DESKTOP-U96FG97 MINGW64 /d/Praktikum_ASD/2025573010095_StrukturDatadanAlgoritma (main)
$ git add praktikum-5/
warning: in the working copy of 'praktikum-5/package.json', LF will be replaced
by CRLF the next time Git touches it
```

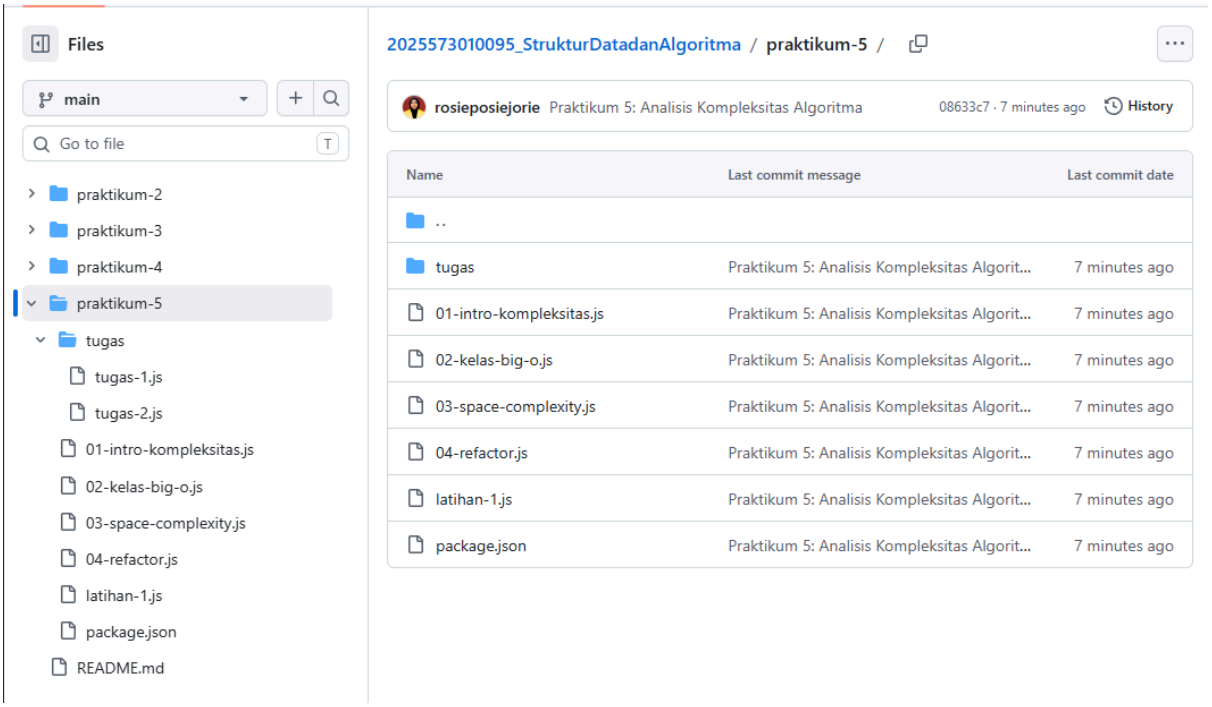
Gambar 18. Tampilan git Add file praktikum-4 di Git Bash

3.2.9.3. Push ke repository GitHub.

```
tasbi@DESKTOP-U96FG97 MINGW64 /d/Praktikum_ASD/2025573010095_StrukturDatadanAlgoritma (main)
$ git commit -m "Praktikum 5: Analisis Kompleksitas Algoritma"
[main 08633c7] Praktikum 5: Analisis Kompleksitas Algoritma
 8 files changed, 389 insertions(+)
 create mode 100644 praktikum-5/01-intro-kompleksitas.js
 create mode 100644 praktikum-5/02-kelas-big-o.js
 create mode 100644 praktikum-5/03-space-complexity.js
 create mode 100644 praktikum-5/04-refactor.js
 create mode 100644 praktikum-5/latihan-1.js
 create mode 100644 praktikum-5/package.json
 create mode 100644 praktikum-5/tugas/tugas-1.js
 create mode 100644 praktikum-5/tugas/tugas-2.js

tasbi@DESKTOP-U96FG97 MINGW64 /d/Praktikum_ASD/2025573010095_StrukturDatadanAlgoritma (main)
$ git push origin main
info: please complete authentication in your browser...
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 4 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 5.18 KiB | 1.73 MiB/s, done.
Total 12 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/rosieposiejorie/2025573010095_StrukturDatadanAlgoritma.git
   c16d94f..08633c7  main -> main
```

Gambar 19. Tampilan git push file praktikum-4 di Git Bash



Gambar 20. File Berhasil di Push ke Repository.

BAB IV
HASIL DAN PEMBAHASAN

Pada praktikum ini dilakukan pengujian dan analisis terhadap beberapa algoritma untuk memahami konsep kompleksitas waktu (*time complexity*) dan kompleksitas ruang (*space complexity*). Pengujian dilakukan dengan membandingkan beberapa fungsi yang memiliki pola perulangan berbeda serta mengukur waktu eksekusinya menggunakan `Date.now()`.

Pada **Latihan 1**, dilakukan identifikasi kompleksitas dari beberapa fungsi. Fungsi A memiliki kompleksitas O(1) karena hanya melakukan satu operasi sederhana. Fungsi B memiliki kompleksitas $O(n^2)$ karena menggunakan dua perulangan bersarang. Fungsi C memiliki kompleksitas O(log n) karena nilai iterasi bertambah secara eksponensial (dikalikan 2). Sedangkan fungsi D memiliki kompleksitas $O(n^3)$ karena terdiri dari tiga perulangan bersarang. Dari hasil tersebut terlihat bahwa semakin banyak perulangan bersarang, maka waktu eksekusi meningkat secara signifikan.

Pada **Latihan 3**, dilakukan perbandingan antara algoritma pencarian pasangan angka dengan dua pendekatan, yaitu metode lambat menggunakan nested loop dan metode cepat menggunakan struktur data Set. Metode lambat memiliki kompleksitas $O(n^2)$ dengan penggunaan memori O(1), sedangkan metode cepat memiliki kompleksitas O(n) dengan penggunaan memori O(n). Hasil pengujian menunjukkan bahwa metode cepat jauh lebih efisien untuk data berukuran besar, meskipun membutuhkan memori tambahan. Hal ini menunjukkan adanya *time-space trade-off* dalam pemilihan algoritma.

Pada **Tugas 1**, dilakukan analisis dan refactoring beberapa fungsi. Pada kasus intersection array, penggunaan Set dapat menurunkan kompleksitas dari $O(n^2)$ menjadi O(n). Pada kasus pengelompokan anagram, kompleksitas yang diperoleh adalah O(n log n) karena proses sorting pada setiap kata. Sedangkan pada kasus pencarian jumlah dua elemen yang sama dengan kuadrat elemen lain, metode lambat memiliki kompleksitas $O(n^3)$, dan metode yang lebih optimal menggunakan sorting serta teknik dua pointer memiliki kompleksitas O(n log n). Hasil ini menunjukkan bahwa pemilihan metode yang tepat dapat meningkatkan efisiensi secara signifikan.

Pada **Tugas 2**, dilakukan benchmarking terhadap beberapa fungsi dengan kompleksitas berbeda, yaitu O(1), O(n), O(n log n), dan $O(n^2)$. Hasil pengujian menunjukkan bahwa fungsi dengan kompleksitas O(1) memiliki waktu eksekusi yang relatif stabil meskipun ukuran data bertambah. Fungsi O(n) meningkat secara linear, sedangkan O(n log n) meningkat lebih cepat namun masih tergolong efisien. Sementara itu, fungsi $O(n^2)$ menunjukkan peningkatan waktu yang sangat drastis. Hasil ini sesuai dengan teori Big O yang menyatakan bahwa semakin tinggi kompleksitas algoritma, maka semakin besar pula pertumbuhan waktu eksekusinya.

BAB V

PENUTUP

5.1. Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa analisis kompleksitas algoritma sangat penting dalam menentukan efisiensi suatu program. Big O Notation digunakan untuk menggambarkan pertumbuhan waktu dan memori terhadap ukuran input.

Algoritma dengan kompleksitas kecil seperti O(1) dan O(log n) memiliki performa yang sangat baik, sedangkan algoritma dengan kompleksitas tinggi seperti $O(n^2)$ dan $O(n^3)$ cenderung tidak efisien untuk data berukuran besar. Selain itu, penggunaan struktur data seperti Set dan Map dapat membantu mengoptimalkan performa algoritma dengan mengurangi kompleksitas waktu, meskipun terkadang membutuhkan tambahan memori.

Dengan demikian, pemilihan algoritma dan struktur data yang tepat sangat berpengaruh terhadap kinerja program secara keseluruhan.

5.2. Saran

Dalam pengembangan program, disarankan untuk selalu mempertimbangkan kompleksitas algoritma agar aplikasi dapat berjalan secara optimal, terutama pada data berukuran besar. Selain itu, mahasiswa diharapkan lebih sering berlatih dalam menganalisis dan membandingkan algoritma untuk meningkatkan pemahaman terhadap efisiensi program.

DAFTAR PUSTAKA

Git SCM. 2024. *Dokumentasi Git*. Diakses dari https://git-scm.com

OpenAI. 2026. *ChatGPT (Large Language Model)*. https://chat.openai.com

Modul Praktikum Algoritma dan Struktur Data. 2026. *Modul 5: ANALISIS KOMPLEKSITAS ALGORITMA*. https://drive.google.com/file/d/1xg7aTVUsfqBn6ievmPEkEpyKfNItx-dH/view